plsa_albertine_v1

August 29, 2024

1 PLSA

Probabilistic latent semantic analysis

1.1 Preliminaries

Import dependencies

```
[ ]: import sys
     import matplotlib.pyplot as plt # type: ignore
```

Set the plotting environment

```
[ ]: %matplotlib inline
```

Put the actual `plsa` package onto the *python path*

```
[ ]: sys.path.append('/Users/leo/stagear/clustering/PLSA/plsa')
```

Import main classes from the `plsa` package

```
[ ]: from plsa import Corpus, Pipeline, Visualize, preprocessors
     from plsa.pipeline import DEFAULT_PIPELINE
     from plsa.algorithms import PLSA
```

1.2 Data Sources

As they can be quite large, no actual text corpus is included with the `plsa` package. Two nice examples to play with could be - Economic News Article Tone and Relevance - Blog Authorship Corpus We are assuming here that you have downloaded one of them (or both) and placed them under a `data` folder under the root of your clone of the `PLSA` GitHub repository.

```
[ ]: csv_file = '/Users/leo/stagear/corpus/laprison_lemma.csv'
     #directory = '../data/blogs'
```

1.3 Set Up the Corpus

Define pre-processing pipeline Depending on there source, actual, real-world text documents are "dirty", and need to be "cleaned up" through a series of pre-processing steps. The `plsa` submodule `preprocessors` contains several of them (see the API documentation). For convenience, they are assembled into a default pipeline that should help you to get some results out-of-the-box.

[ ]:
```
pipeline = Pipeline(*DEFAULT_PIPELINE)
pipeline = Pipeline(preprocessors.remove_non_ascii, preprocessors.to_lower,␣
 ↪preprocessors.remove_numbers, preprocessors.tokenize)

#pipeline = Pipeline(preprocessors.remove_non_ascii, preprocessors.to_lower,␣
 ↪preprocessors.remove_numbers, preprocessors.remove_punctuation,␣
 ↪preprocessors.tokenize)
pipeline
```

[ ]:
```
Pipeline:
========
0: remove_non_ascii
1: to_lower
2: remove_numbers
3: tokenize
```

Load corpus Execute either this cell ...

[ ]:
```
corpus = Corpus.from_csv(csv_file, pipeline, max_docs=10000)
corpus
```

[ ]:
```
Corpus:
======
Number of documents: 5815
Number of words:     4968
```

1.4 Run PLSA

Choose the number of topics

[ ]:
```
n_topics = 20
```

Instantiate a PLSA model

[ ]:
```
plsa = PLSA(corpus, n_topics, True)
plsa
```

[ ]:
```
PLSA:
====
Number of topics:     20
Number of documents:  5815
Number of words:      4968
Number of iterations: 0
```

Notice that we did not do any iterations yet.

Fit a PLSA model

[ ]:
```
result = plsa.fit(max_iter=80)
plsa
```

[ ]:
```
PLSA:
====
Number of topics:     20
Number of documents:  5815
Number of words:      4968
Number of iterations: 85
```

Now we indeed did do some iterations.

Find the best PLSA model of many As with any iterative algorithm, also the probabilities in PSLA need to be (randomly) initialized prior to the first iteration step. Therefore, calling the `fit` method of two different `PLSA` instances operating on the *same* corpus with the *same* number of topics potentially leads to (slightly) different results, corresponding to different local minima of the Kullback-Leibler divergence between the true document-word probability and its approximate factorization. To mitigate this effect, perform multiple runs and pick the best model.

Be patient, this may take a while …

[ ]:
```
#result = plsa.best_of(5)
```

Examine the results Feel free to explore the attributes of the `result` object. See the API documentation for more information.

For example, we could see the relative prevalence of the individual topics we found.

[ ]:
```
result.topic
```

[ ]:
```
array([0.0609685 , 0.05907762, 0.05820016, 0.05594122, 0.05202501,
       0.05200577, 0.0516431 , 0.0513877 , 0.05060018, 0.05030736,
       0.04950622, 0.04925576, 0.04887463, 0.047115  , 0.0467272 ,
       0.0451925 , 0.04444131, 0.04315083, 0.04199277, 0.04158715])
```

And, of course, we can look at individual topics, that is, how important which word is for which topic. Let's look at the top-10 words of the first topic.

[ ]:
```
result.word_given_topic[0][:10]
```

[ ]:
```
(('dire', 0.08666664135842035),
 ('!', 0.07225757207735323),
 ('savoir', 0.04933144632756951),
 (':', 0.04830927522144571),
 ('il', 0.03059479675778023),
 ('vous', 0.02568852563554567),
 ('ah', 0.022645853385958804),
 ('penser', 0.021642921100862057),
 ('trop', 0.021548833356970477),
 ('je', 0.020783151435880252))
```

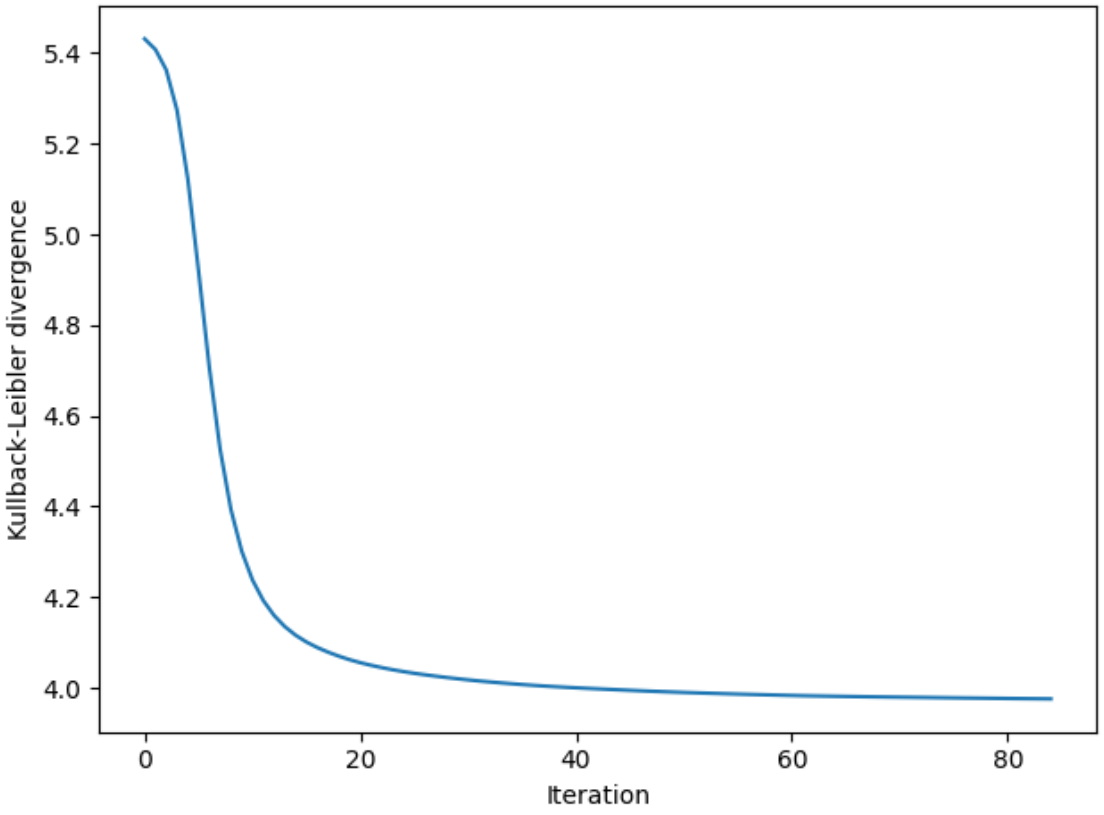
1.5 Visualize the Results

[ ]:
```
visualize = Visualize(result)
visualize
```

[ ]:
```
Visualize:
=========
Number of topics:    20
Number of documents: 5815
Number of words:     4968
```

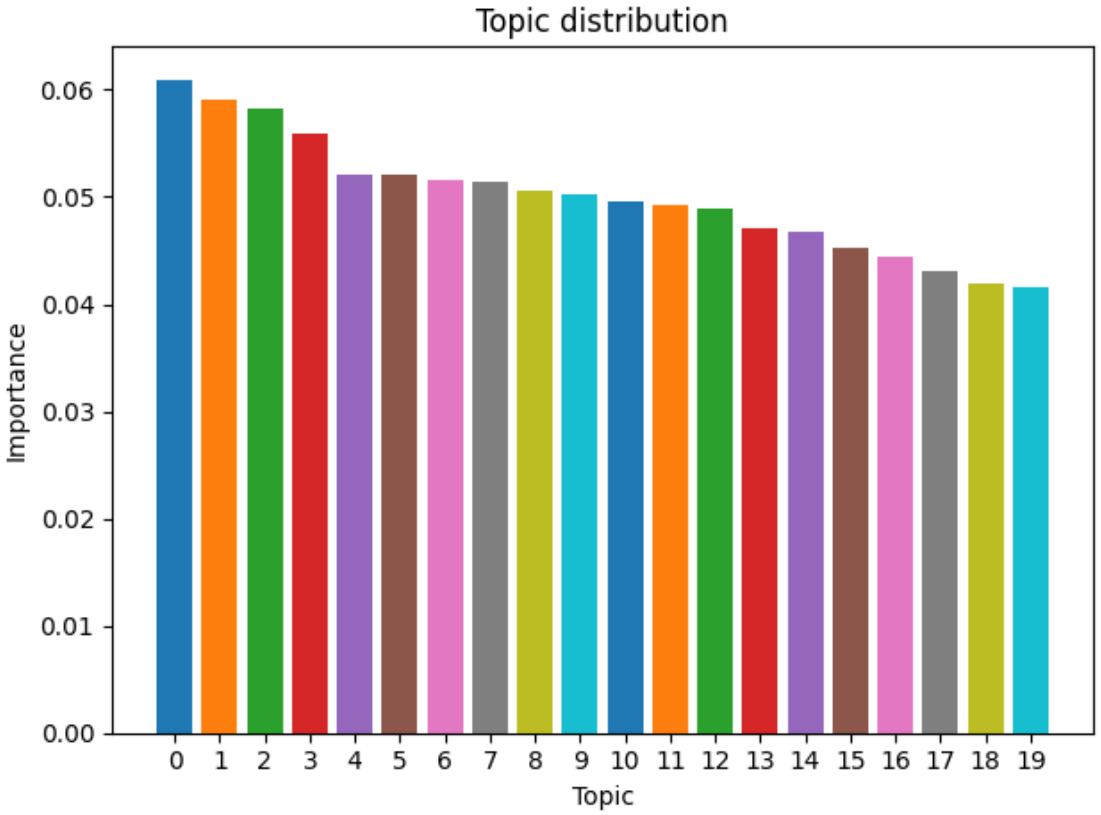
Convergence Since PLSA uses an iterative expectation-maximization (EM) style algorithm, let's make sure we have achieved reasonable convergence.

[ ]:
```
fig, ax = plt.subplots()
_ = visualize.convergence(ax)
fig.tight_layout()
```



Relative topic importance How important are the topics we found in the corpus?

[ ]:
```
fig, ax = plt.subplots()
_ = visualize.topics(ax)
fig.tight_layout()
```

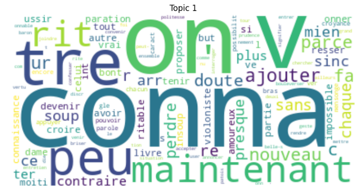


The topics The most interesting part is probably the topics themselves, We can visualize them as word clouds.

[ ]:
```
fig = plt.figure(figsize=(40, 40))
_ = visualize.wordclouds(fig)
```

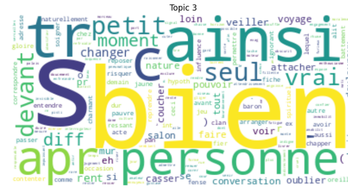
Topic 0

Topic 1



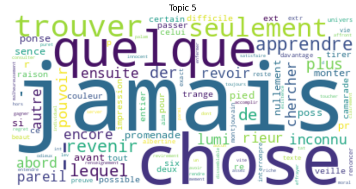
Topic 2

Topic 3



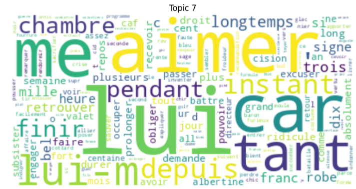
Topic 4

Topic 5



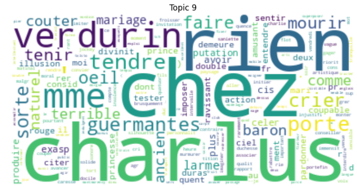
Topic 6

Topic 7

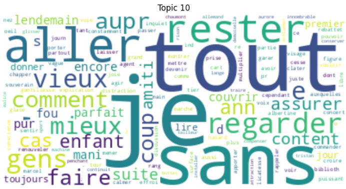


Topic 8

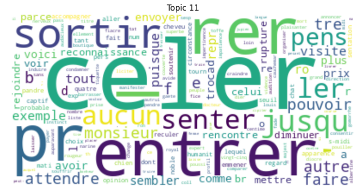
Topic 9



Topic 10



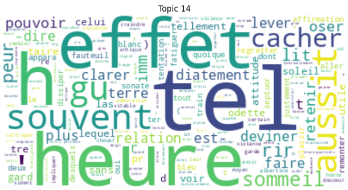
Topic 11



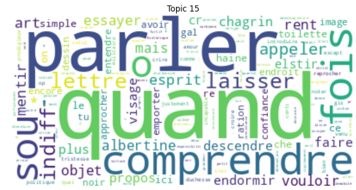
Topic 12

Topic 13

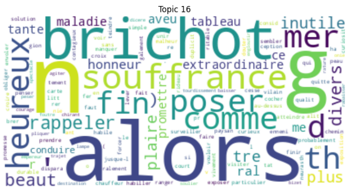
Topic 14



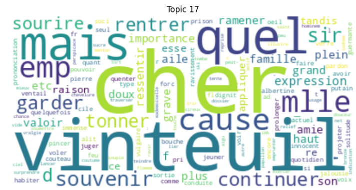
Topic 15



Topic 16

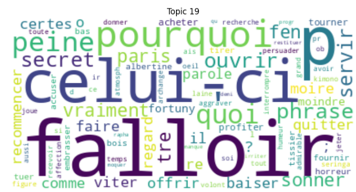


Topic 17



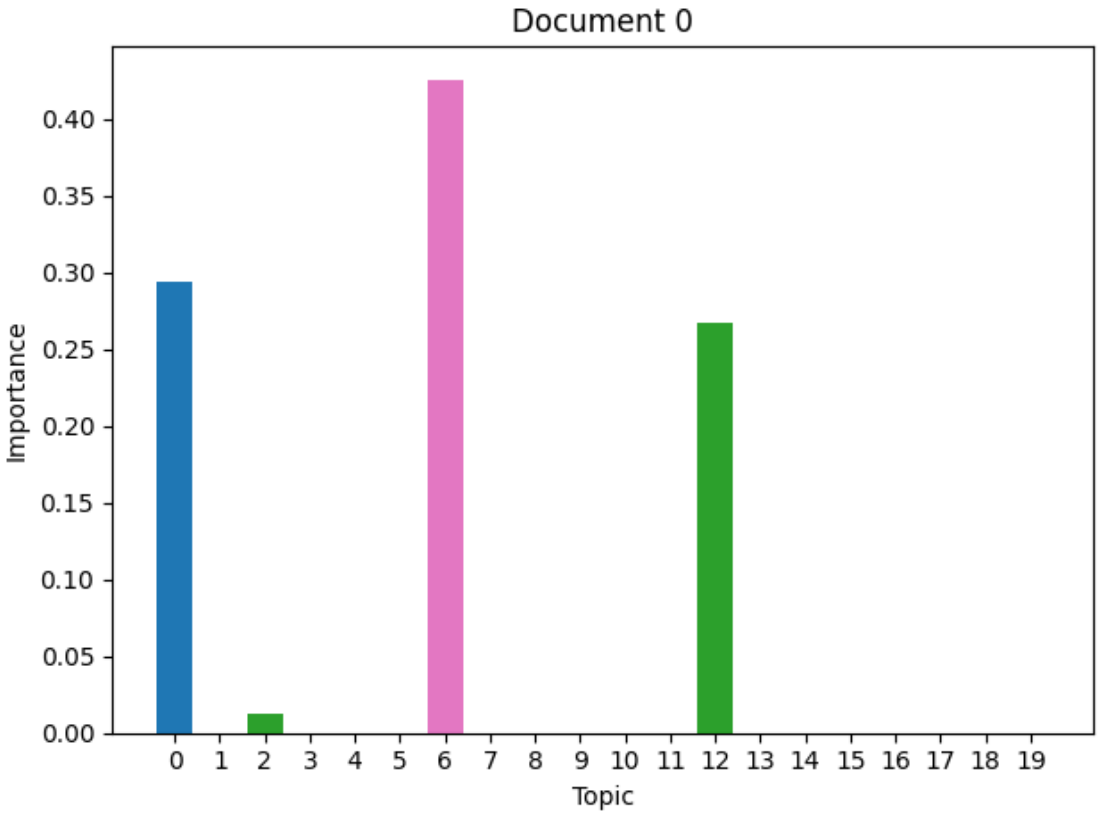
Topic 18

Topic 19



Relative topic importance in a document Also interesting is the mixture of topics in each document. Let's look at the first one.

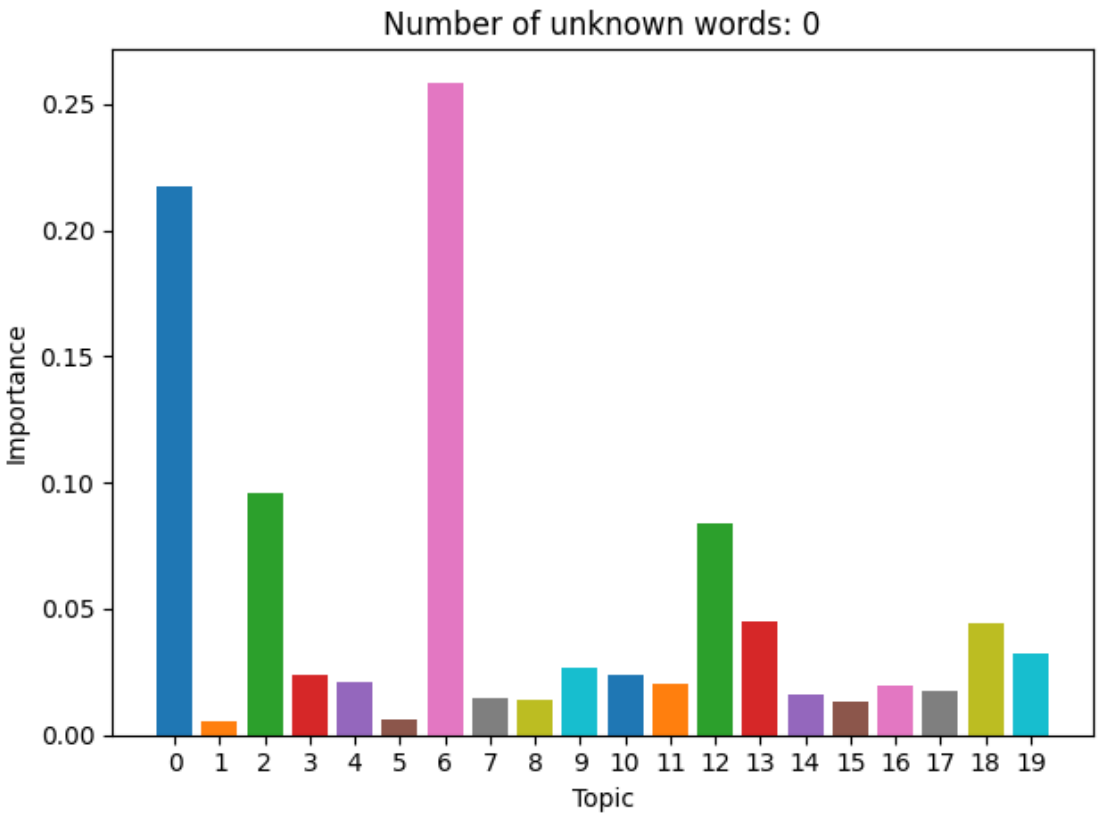
[ ]:
```
fig, ax = plt.subplots()
_ = visualize.topics_in_doc(0, ax)
fig.tight_layout()
```



Let's compare this with what the prediction would look like, pretending that this document wasn't seen before.

[ ]:
```
for first in corpus.raw:
    if first:
        break

fig, ax = plt.subplots()
_ = visualize.prediction(first, ax)
fig.tight_layout()
```



Similar, but not quite the same. This is the very nature of matrix factorization algorithms, to which PLSA can be seen to belong. We try to approxmiate the original counts of each word in each document with a lower-dimensional representation of the data. That's why the topic composition get's somewhat "blurred".

Prediction for a new document We can also visualize the predicited topic composition for a new document.

[ ]:
```
# new_doc = 'Hello! This is the federal humpty dumpty agency for state funding.'

# fig, ax = plt.subplots()
# _ = visualize.prediction(new_doc, ax)
# fig.tight_layout()
```

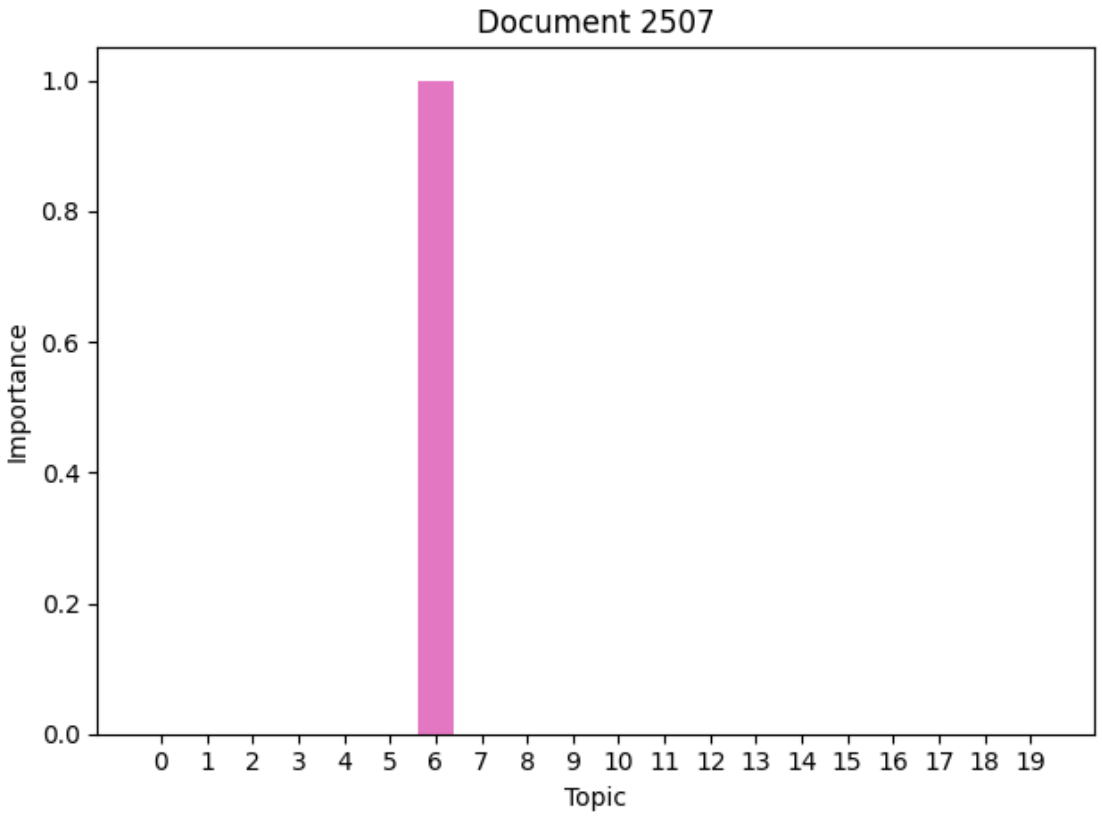
[ ]:
```
# for i in range(0, 19):
#     visualize.words_in_topic(i, plt.axes._subplots.AxesSubplot)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[25], line 2
      1 for i in range(0, 19):
```

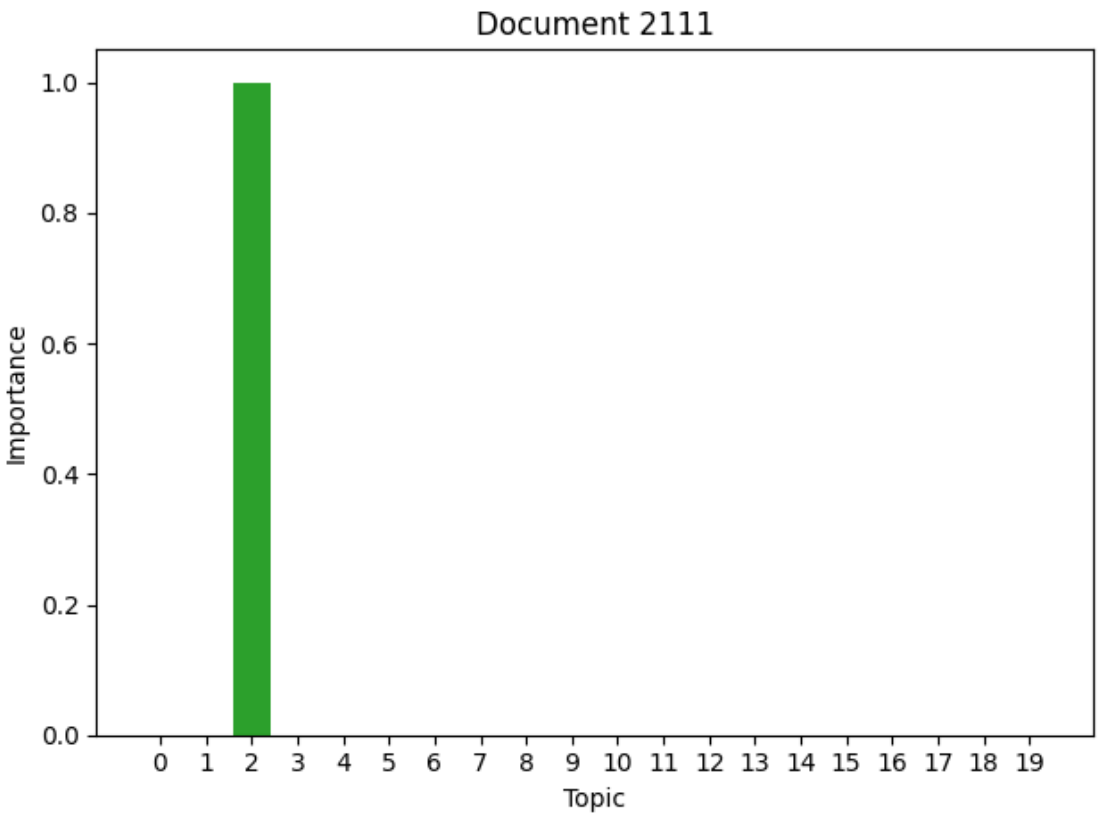
```
----> 2     visualize.words_in_topic(i, plt.axes._subplots.AxesSubplot)

AttributeError: 'function' object has no attribute '_subplots'
```

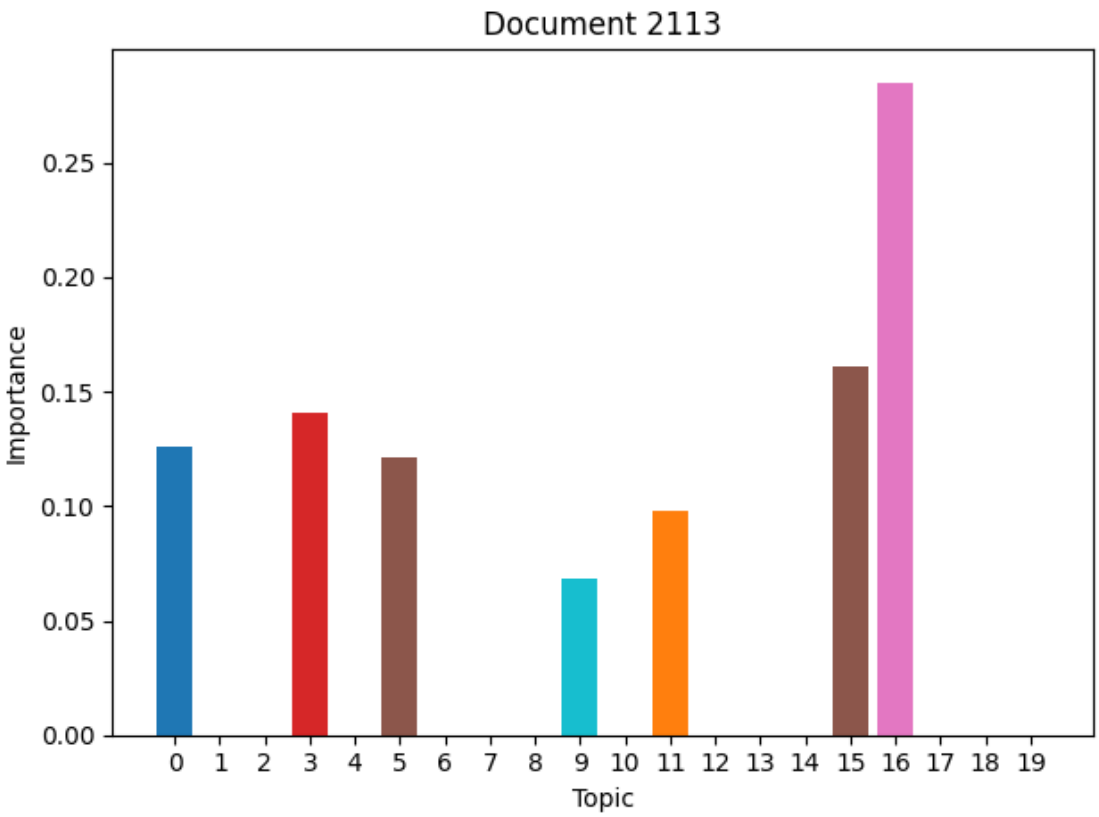
[ ]:
```
fig, ax = plt.subplots()
_ = visualize.topics_in_doc(2507, ax)
fig.tight_layout()
```



[ ]:
```
fig, ax = plt.subplots()
_ = visualize.topics_in_doc(2111, ax)
fig.tight_layout()
```



[ ]:
```
fig, ax = plt.subplots()
_ = visualize.topics_in_doc(2113, ax)
fig.tight_layout()
```


Document 2113
Importance
0.25
0.20
0.15
0.10
0.05
0.00
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
Topic